

Improved Architectures for a Floating-Point Fused Dot Product Unit

Jongwook Sohn and Earl E. Swartzlander, Jr.
 Department of Electrical and Computer Engineering
 The University of Texas at Austin
 Austin, TX 78712

Abstract—This paper presents improved architectures for a floating-point fused two-term dot product unit. The floating-point fused dot product unit is useful for a wide variety of digital signal processing (DSP) applications including complex multiplication and fast Fourier transform (FFT) and discrete cosine transform (DCT) butterfly operations. In order to improve the performance, a new alignment scheme, early normalization, a four-input leading zero anticipation (LZA), a dual-path algorithm, and pipelining are applied. The proposed designs are implemented for single precision and synthesized with a 45nm standard cell library. The proposed dual-path design reduces the latency by 25% compared to the traditional floating-point fused dot product unit. Based on a data flow analysis, the proposed design can be split into three pipeline stages. Since the latencies of the three stages are fairly well balanced, the throughput is increased by a factor of 2.8 compared to the non-pipelined dual-path design.

Keywords—Digital signal processing (DSP), two-term dot product unit, floating-point arithmetic, floating-point fused operations, high-speed computer arithmetic.

I. INTRODUCTION

Floating-point operations are widely used for advanced applications such as 3D graphics, signal processing, and scientific computations. These require a wide dynamic range. Fixed-point arithmetic is not sufficient for this, but floating-point arithmetic, such as that which is specified in IEEE-754 Standard for floating-point arithmetic [1], can represent a wide range of numbers from tiny fractional numbers to nearly infinitely huge numbers so that the overflow and underflow are avoided. However, the floating-point operations require complex processes like alignment, normalization and rounding, which significantly increase the area, power consumption and latency. One solution is to merge or “fuse” several operations in one floating-point unit to reduce the area, power and latency by sharing the common logic of the operations. In order to improve the floating-point units, several fused units have been introduced: Fused multiply-add (FMA) [2]–[4], fused add-subtract [5], [6], and fused dot product [7].

This paper presents improved architecture designs and implementation details for a floating-point fused two-term dot product unit. The floating-point fused dot product unit is useful for many digital signal processing (DSP) applications [8], [9]. Therefore, the improved floating-point fused dot product unit will contribute to the next generation of floating-point unit designs and DSP application development.

The proposed floating-point fused two-term dot product unit takes four normalized operands and computes the sum or difference of the two products as

$$P = AB \pm CD. \quad (1)$$

It supports all five rounding modes specified in the IEEE-754 Standard [1]. Several techniques are employed not only to improve the performance but also to reduce the area and power consumption:

- 1) For fast alignment, a new alignment scheme is proposed. By swapping the significands and shifting only the smaller significands, the shift amount is reduced so that the area and latency are reduced.
- 2) Early normalization is applied, which was proposed to reduce the latency of the fused multiply-add unit [4]. By performing the normalization prior to the addition, the length of significands can be reduced using the sticky logic, reducing the addition size by half. The sign is also determined prior to the addition so that the addition and rounding can be performed together, which significantly reduces the latency.
- 3) Since the normalization is performed prior to the addition, the leading zero anticipation (LZA) and normalization shift are on the critical path. In order to reduce the latency, a four-input LZA is proposed.
- 4) The dual-path algorithm is employed to improve the performance. The dual-path logic consists of a far path and a close path. Based on the exponent difference, a path is selected. In the far path logic, massive cancellation does not occur so that LZA and normalization are unnecessary. In the close path logic, only a two bit significand alignment is required so that the large significand shifter is unnecessary. By removing the unnecessary logic in each path, the latency is reduced.
- 5) In order to increase the throughput, pipelining can be applied. Based on the data flow analysis, the proposed dual-path floating-point fused dot product unit is split into three stages. Since the latencies of three stages are relatively well balanced, the throughput is improved.

Section II presents the traditional design for a floating-point fused two-term dot product unit. The next three sections present the improved designs and implementations for the floating-point fused dot product unit. In Section III, the enhanced design for a floating-point fused dot product unit using the new alignment, early normalization, fast rounding and the four-input LZA is introduced. A dual-path design

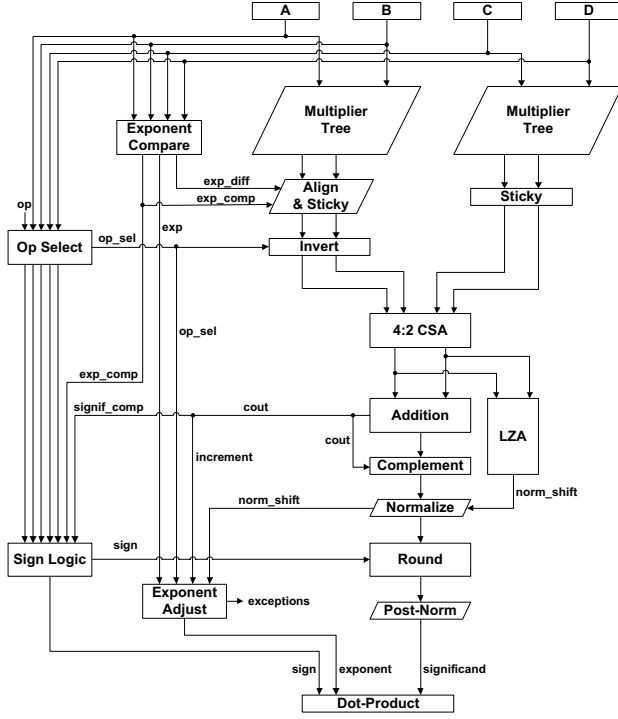


Figure 1. Traditional floating-point fused dot product unit (after [7], [8]).

and its implementation details are presented in Section IV. The pipelined dual-path design based on the data flow analysis is presented in Section V. All the proposed designs are implemented for single precision. The implementations are evaluated in terms of the latency, area and power consumption and the results are presented in Section VI.

II. THE TRADITIONAL FUSED DOT PRODUCT UNIT

The floating-point dot product unit can be simply implemented by using two floating-point multipliers and a floating-point adder. However, such a discrete version requires large area, power consumption and latency. Moreover, since rounding is performed three times (after each of the multiplications and after the addition), the accuracy is decreased. In order to reduce the area and latency, and increase the accuracy, the floating-point fused dot product unit is introduced. It shares the common logic such as exponent compare, significand addition, exponent adjust and sign logic so that the area, power consumption and latency are reduced. Also, the floating-point fused dot product unit performs only a single rounding so that the accuracy increases.

The traditional floating-point fused dot product unit [7], [8] is based on the floating-point fused multiply-add unit as shown in Figure 1. The steps to execute the floating-point fused dot product are

- 1) Four floating-point numbers are unpacked into their signs, exponents and significands.
- 2) Two multiplier trees are used to produce two pairs of sums and carries (a total of four numbers). In parallel, two sums of exponents are computed and compared to determine the greater product and the difference is com-

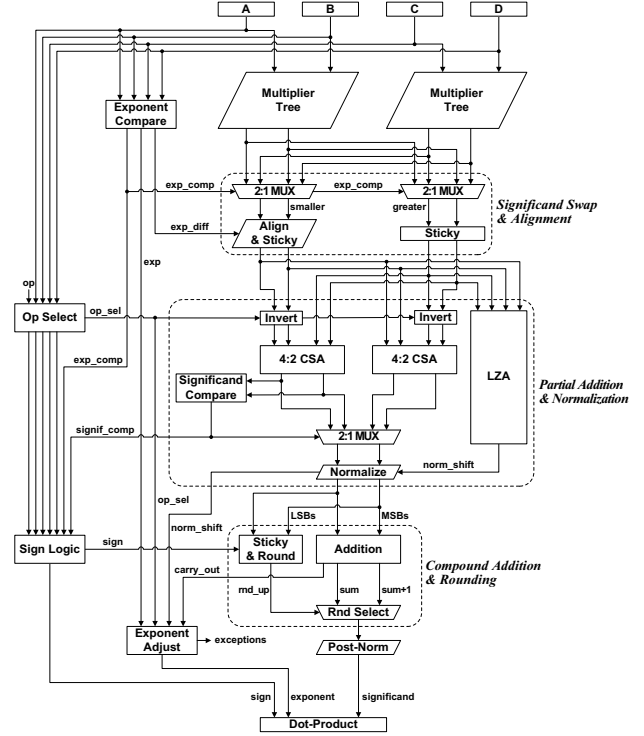


Figure 2. Enhanced floating-point fused dot product unit.

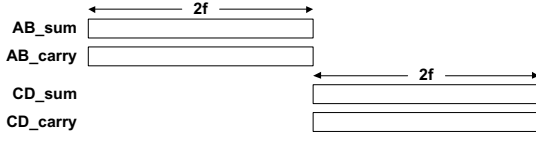
puted. Also, the operation (addition or subtraction) is selected using the sign bits and op code.

- 3) One sum and carry pair is aligned based on the exponent difference result and inverted if the operation is subtraction. The two pairs of significands are passed to a 4:2 reduction tree. Carry save adders are used to form the reduction tree, which reduces the four significands to two.
- 4) The two significands are summed and complemented if the sum is negative. LZA is performed for fast normalization. The significand comparison result is passed to the sign logic so that the sign is determined.
- 5) Since some of rounding modes specified in IEEE-754 Standard [1] require the sign (i.e., round to positive and negative infinity), the sign logic must be performed prior to the round logic.
- 6) The normalized significands are rounded and post-normalized. The exponent is adjusted with the addition carry out and the normalization shift amount.

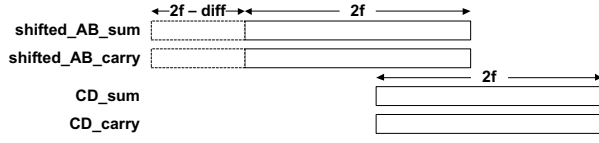
III. AN ENHANCED FUSED DOT PRODUCT UNIT

The traditional floating-point fused dot product unit reduces the area, latency and power consumption compared to the discrete floating-point dot product unit [7], [8]. However, it is an initial design so that more optimizations can be applied to improve the performance. In this section, several optimizations are proposed to improve the floating-point fused dot product unit: 1) A new alignment scheme, 2) Early normalization and fast rounding scheme, and 3) Four-input LZA. Figure 2 shows the enhanced floating-point fused dot product unit.

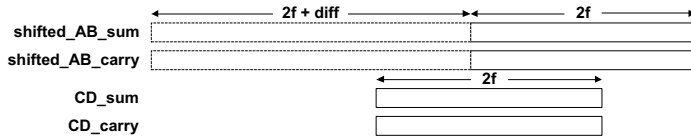
- Before alignment



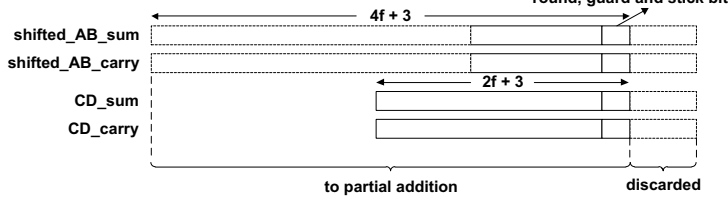
- If $AB > CD$



- If $AB < CD$



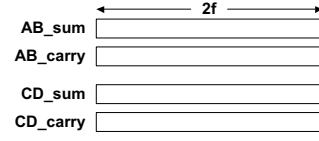
- After alignment



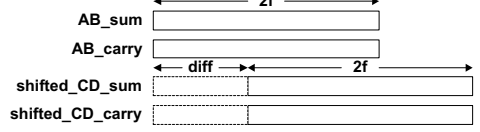
* f = number of significand bits
diff = exponent difference

(a)

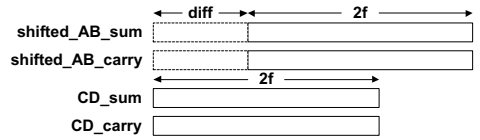
- Before alignment



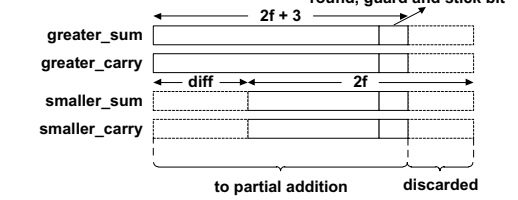
- If $AB > CD$



- If $AB < CD$



- After alignment



(b)

Figure 3. Alignment comparison: (a) The traditional alignment scheme, (b) The new alignment scheme.

A. A New Alignment Scheme

The traditional floating-point fused dot product unit performs the significand alignment on a single side significand pair (sum and carry) as shown in Figure 3(a). The one way alignment requires a large shift amount, which increases the latency of the critical path. In order to reduce the latency of the alignment, the new alignment scheme swaps the significands to shift the smaller significand pair so that the shift amount is reduced as shown in Figure 3(b). Also, the sticky logic is performed to generate the round, guard and sticky bits. Then, the LSBs under the sticky bit can be discarded so that the length of the significand pairs is reduced. If the exponent difference is larger than 2, massive cancellation does not occur so that the discarded bits are not affected by the normalization. If the exponent difference is 2 or less, the shifted bits are maintained by the round, guard and sticky bits. Due to the reduced shift amount and sticky logic, smaller significand pairs are generated compared to the traditional alignment, resulting in reduced area and power consumption for the following logic.

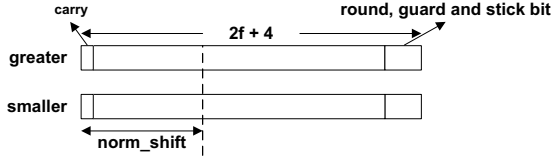
B. Early Normalization and Fast Rounding Scheme

If the operation is subtraction, the significand pairs are inverted. The inverted and non-inverted significand pairs are passed to each reduction tree and reduced to two signifi-

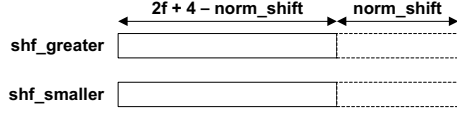
cands. The two significands are compared and the comparison result selects the one that is not to be complemented after the significand addition. Also, the significand comparison result is used in the sign logic.

The reduced significand pair is passed to the normalization. The traditional floating-point fused dot product unit performs the normalization after the significand addition, which requires a large adder and complement followed by the round logic. For fast significand addition and rounding, early normalization is applied, which was previously proposed for the fused multiply-add unit [4]. By normalizing the significands prior to the significand addition, the length of the adder can be reduced up to the length of the final significand and the round logic can be performed in parallel. Figure 4 shows the early normalization and sticky logic. The MSBs of the normalized significands are passed to the addition and the LSBs are passed to the sticky and round logic. The sticky logic is performed again to generate round, guard and sticky bits. The 1st and 2nd bits under the LSB become the guard and round bits and the sticky bit is set if at least one bit of the rest of the LSBs is 1, which can be implemented with OR trees. The four bits including the LSB, guard, round and sticky bits are used for the round logic to simplify the round logic and the rest of the LSBs are discarded.

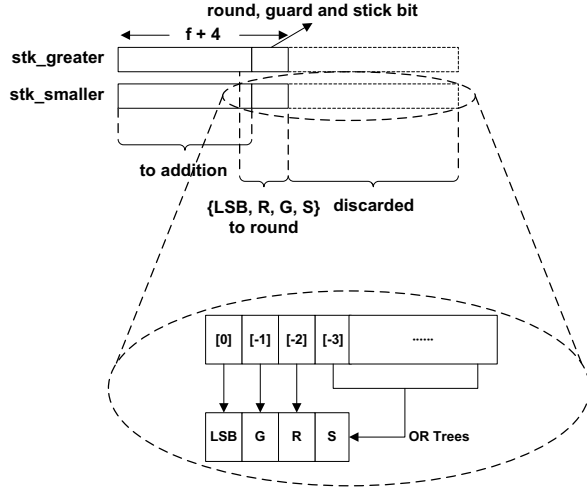
- **After reduction tree**



- **Normalization**



- **Sticky**



* f = number of significand bits

Figure 4. Early normalization and sticky logic.

Since some of round modes specified in IEEE-754 Standard [1] require knowing the sign (i.e., round to positive and negative infinity), the sign logic must be performed prior to the round logic. The significand comparison result from the partial addition is used for the sign logic, if the exponent difference is zero. The sign bit is passed to the final result as well as the round logic. For fast rounding, compound addition is used, which produces the rounded and unrounded sums together and the round logic selects the correct result. By performing the significand addition and rounding together, the latency is significantly reduced.

C. Four-Input LZA

Since the normalization is performed prior to the significand addition, the LZA and normalization is placed on the critical path. To use the traditional two-input LZA for the floating-point fused dot product unit, a 4:2 reduction tree is required prior to the LZA. The four-input LZA reduces the overhead of the reduction tree by encoding the four inputs at once.

TABLE I. FOUR-INPUT LZA PRE-ENCODING PATTERNS FOR $W > 0$

W vector	Leading Zeros	Pre-encoding Pattern
$0^k 11(x)$	k	$e_{f-1} g_i g_{i+1}$
$0^k 10(1 \text{ or } 0)$	k	$e_{f-1} g_i e_{i+1}$
$0^k 10^l(\bar{1})$	$k + 1$	$e_{f-1} g_i e_{i+1}^*$
$0^k 1 \bar{1}^l 1(x)$	$k + l$	$\bar{e}_{f-1} s_i g_{i+1}$
$0^k 1 \bar{1}^l 0(1 \text{ or } 0)$	$k + l$	$\bar{e}_{f-1} s_i e_{i+1}$
$0^k 1 \bar{1}^l 0^m(\bar{1})$	$k + l + 1$	$\bar{e}_{f-1} s_i e_{i+1}^*$

* Correction needed

The four-input LZA can be implemented by extending the traditional two-input LZA [10]. In order to encode four inputs, the W vector is generated with bitwise operations as

$$W = A + B - C - D$$

$$w_i = a_i + b_i - c_i - d_i, \quad w_i \in \{-2, -1, 0, 1, 2\}, \quad (2)$$

where a_i, b_i, c_i, d_i are the i^{th} bits of the four significands. The W vector can be represented by one of the five elements, $\bar{2}_i, \bar{1}_i, 0_i, 1_i$ and 2_i , indicating w_i equals to $-2, -1, 0, 1$ and 2 , respectively. The W vector is pre-encoded into three symbols, g, e and s as

$$\begin{aligned} g_i &= 1 \text{ if } w_i = 1 \\ e_i &= 1 \text{ if } w_i = 0 \\ s_i &= 1 \text{ if } w_i = \bar{1}. \end{aligned} \quad (3)$$

To handle the cases of -2 and 2 , two consecutive bits are involved for pre-encoding. For example, the bit pattern $0_i 2_{i+1}$ and $1_i \bar{2}_{i+1}$ are considered as $1_i 0_{i+1}$ and $0_i 0_{i+1}$, respectively. Thus, the three symbols are represented as

$$\begin{aligned} g_i &= 2_i(2_{i+1} + \bar{2}_{i+1}) + 1_i(1_{i+1} + 0_{i+1} + \bar{1}_{i+1}) + 0_i 2_{i+1} \\ e_i &= 2_i(1_{i+1} + 0_{i+1} + \bar{1}_{i+1}) + 1_i(2_{i+1} + \bar{2}_{i+1}) \\ &\quad + 0_i(1_{i+1} + 0_{i+1} + \bar{1}_{i+1}) + \bar{1}_i(2_{i+1} + \bar{2}_{i+1}) \\ &\quad + \bar{2}_i(1_{i+1} + 0_{i+1} + \bar{1}_{i+1}) \\ s_i &= 0_i \bar{2}_{i+1} + \bar{1}_i(1_{i+1} + 0_{i+1} + \bar{1}_{i+1}) + \bar{2}_i(2_{i+1} + \bar{2}_{i+1}). \end{aligned} \quad (4)$$

The pre-encoding patterns that terminate the leading zeros and the corresponding leading zeros for $W > 0$ are shown in Table I. The number of leading zeros is computed with the three symbols as

$$f_i(pos) = e_{i-1} g_i \bar{s}_{i+1} + \bar{e}_{i-1} s_i \bar{s}_{i+1} \text{ for } W > 0. \quad (5)$$

Similarly, for the bit patterns when $W < 0$,

$$f_i(neg) = e_{i-1} s_i \bar{g}_{i+1} + \bar{e}_{i-1} g_i \bar{g}_{i+1} \text{ for } W < 0. \quad (6)$$

Combining two equations, the F vector is generated as

$$f_i = e_{i-1}(g_i \bar{s}_{i+1} + s_i \bar{g}_{i+1}) + \bar{e}_{i-1}(s_i \bar{s}_{i+1} + g_i \bar{g}_{i+1}). \quad (7)$$

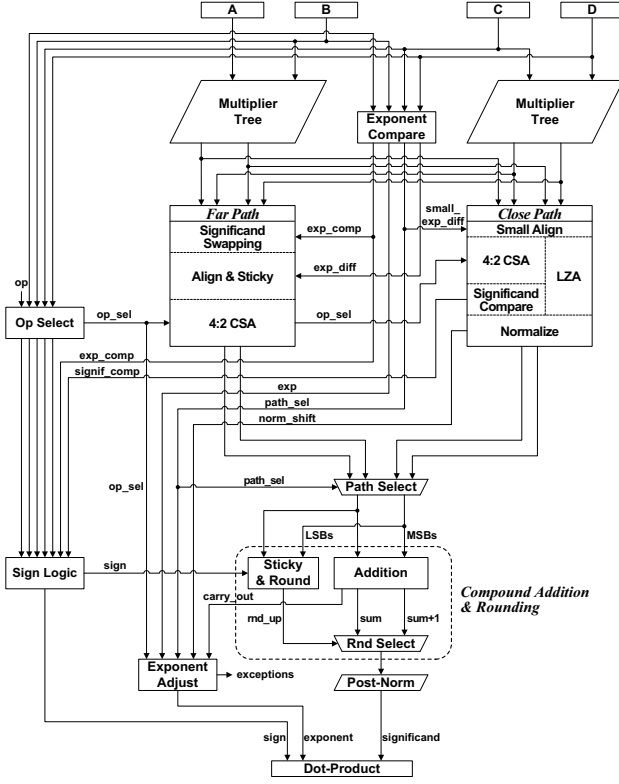


Figure 5. Dual-path floating-point fused dot product unit.

This is essentially the same equation as that of the traditional two-input LZA [10]. The F vector is encoded with the leading zero detector (LZD) to obtain the leading zeros, which is the shift amount of the normalization. For fast normalization, the MSBs of the shift amount are generated so that the LZD tree and the normalization shifter are overlapped [4].

Like most of the two-input LZAs are inexact due to the possible 1 bit error, the proposed four-input LZA also requires correction logic. For fast error detection and correction, concurrent error correction logic can be used, which was previously proposed [10]–[12]¹. In the cases of the bit patterns² $0^k10^l\bar{1}$ and $0^k110^l\bar{1}$ for $W > 0$ and $0^k\bar{1}0^l1$ and $0^k\bar{1}10^l1$ for $W < 0$, correction is required by adding 1.

IV. A DUAL-PATH FUSED DOT PRODUCT UNIT

To achieve a high speed floating-point fused dot product unit, the dual-path approach is employed. The dual-path floating-point fused dot product unit consists of a far path and a close path as shown in Figure 5. The path is determined based on the exponent difference. The far path skips the LZA and normalization and the close path skips the significand swap and alignment. Since these two processes are

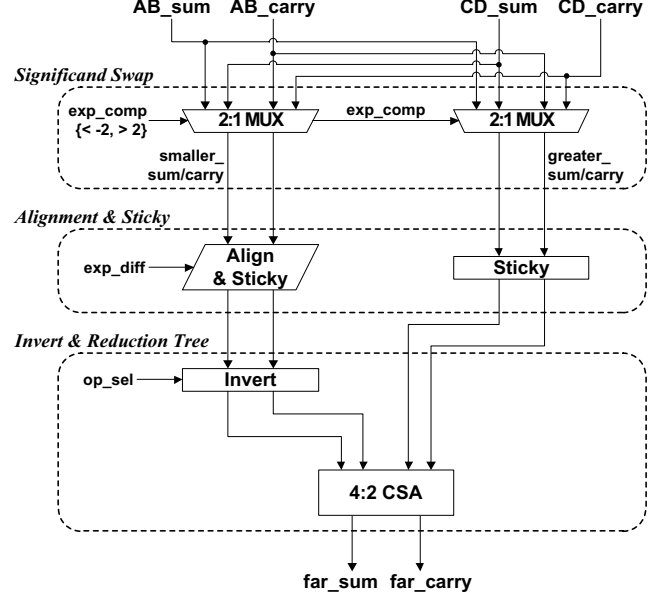


Figure 6. Far path logic for a dual-path fused dot product unit.

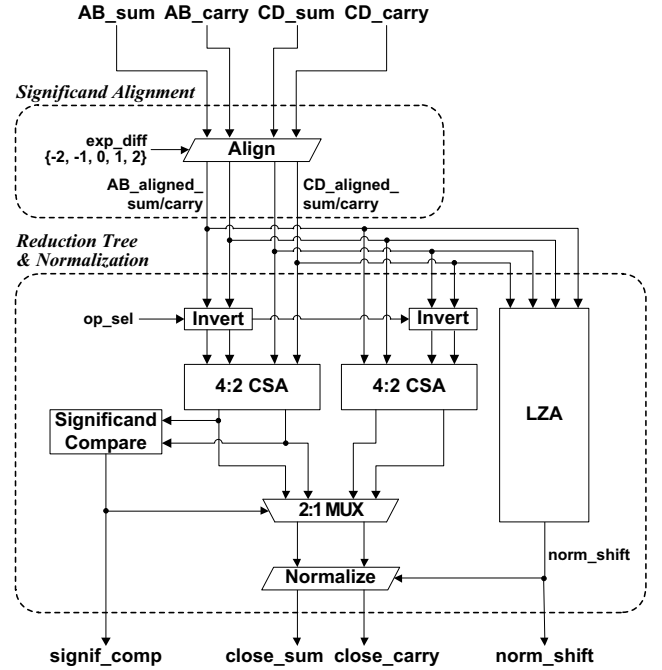


Figure 7. Close path logic for a dual-path fused dot product unit.

the bottlenecks of the traditional floating-point fused dot product unit, the two path process improves the performance.

A. Far Path Logic

The far path logic for the dual-path floating-point fused dot product unit can be implemented as the significand swap and alignment part of the enhanced floating-point fused dot product unit as shown in Figure 6. The far path is selected if

¹ The error correction logic in [10] is modified by [11] and [12] to improve the accuracy and eliminate the redundancy, respectively.

² The notation x^k denotes a bit string of k consecutive bits, where $x \in \{\bar{1}, 0, 1\}$ and $k \geq 0$.

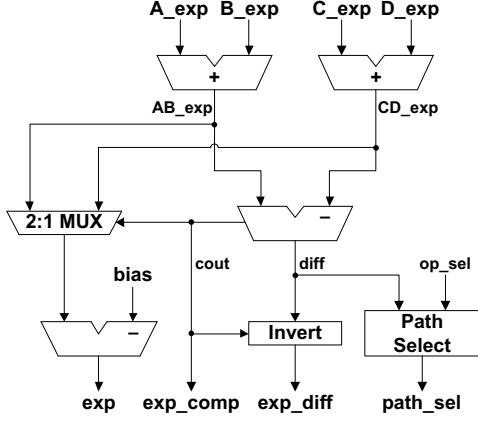


Figure 8. Exponent compare and path select logic.

the exponent difference is larger than 2 or the operation is addition. Since the addition of four significands generates a carry out of up to 3, the exponent difference margin for the far path is two bits, which is 1 bit larger than that of the general dual-path floating-point adder. In this case, massive cancellation during the subtraction does not occur so that the LZA and normalization are unnecessary. Two multiplexers are used to swap the significand pairs so that only the smaller significand pair is aligned, which reduces the shift amount. The aligned significand pair is inverted if the operation is subtraction. The sticky logic is performed for both significand pairs to reduce the significand length. The significand pair for the far path is generated by the reduction tree, which reduces the four significands to two.

B. Close Path Logic

The close path is selected if the exponent difference is less than 3 and the operation is subtraction. In this case, only a two bit shifter is required for the significand alignment. The significand pairs are aligned as

$$AB_{aligned} = \begin{cases} (AB_{signif}, 00) & \text{if } AB_{exp} - CD_{exp} = 0, 1, 2 \\ (0, AB_{signif}, 0) & \text{if } AB_{exp} - CD_{exp} = -1 \\ (00, AB_{signif}) & \text{if } AB_{exp} - CD_{exp} = -2 \end{cases} \quad (8)$$

$$CD_{aligned} = \begin{cases} (CD_{signif}, 00) & \text{if } AB_{exp} - CD_{exp} = 2 \\ (0, CD_{signif}, 0) & \text{if } AB_{exp} - CD_{exp} = 1 \\ (00, CD_{signif}) & \text{if } AB_{exp} - CD_{exp} = 0, -1, -2. \end{cases} \quad (9)$$

The rest of the close path logic can be implemented as the partial addition and normalization part of the enhanced floating-point fused dot product unit as shown in Figure 7.

C. The Other Sub-logic

Among the two significand pairs from the far path and close path, a significand pair is selected based on the exponent difference and the operation. The selected significand pair is passed to the significand addition and round logic. The significand addition and round logic can be implemented similar to that of the enhanced floating-point fused dot product unit which was introduced in the previous section.

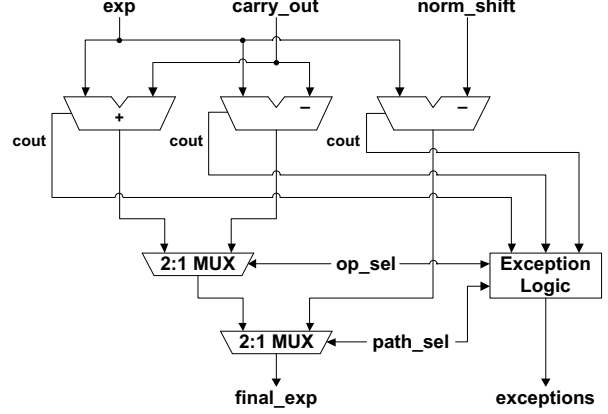


Figure 9. Exponent adjust and exception logic.

This section contains the rest of sub-logic for the floating-point dual-path fused dot product unit.

The operation select logic generates the op_sel bit, which determines if the significands are inverted for the significand subtraction. Using the four sign bits of the operands and the input operator, the operation is selected as

$$op_sel = \begin{cases} AB_{sign} \oplus CD_{sign} & \text{if } op = add \\ AB_{sign} \oplus CD_{sign} & \text{if } op = sub, \end{cases} \quad (10)$$

where AB_{sign} is $A_{sign} \oplus B_{sign}$ and CD_{sign} is $C_{sign} \oplus D_{sign}$.

The exponent compare and path select logic are shown in Figure 8. For the exponent process, two pairs of exponents are summed and a greater exponent sum is selected. Then, the bias is subtracted for the exponent result. The two exponent sums are compared to determine the greater one. The exponent comparison result is used for the significand swapping and the exponent difference is used for the alignment. Also, the path selection bit is determined based on the exponent difference and the operation as

$$path_sel = \begin{cases} 1 & \text{if } |AB_{exp} - CD_{exp}| \leq 2 \text{ or } op_sel = 0 \\ 0 & \text{otherwise.} \end{cases} \quad (11)$$

Figure 9 shows the exponent adjust logic, which adjusts the exponent by adding or subtracting the carry out from the significand addition. Since the four significands generate a carry out of up to 3, two carry out bits are used for the adjustment. The normalization shift amount is subtracted in case of the massive cancellation. Using the selection bits and the carry outs from the addition and subtractions, the exceptions are detected. The three exception cases specified in IEEE-754 Standard [1] are detected as

$$overflow = \begin{cases} 1 & \text{if } exp \geq max_exp \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

$$underflow = \begin{cases} 1 & \text{if } exp \leq 0 \\ 0 & \text{otherwise} \end{cases} \quad (13)$$

$$inexact = overflow || underflow || round_up, \quad (14)$$

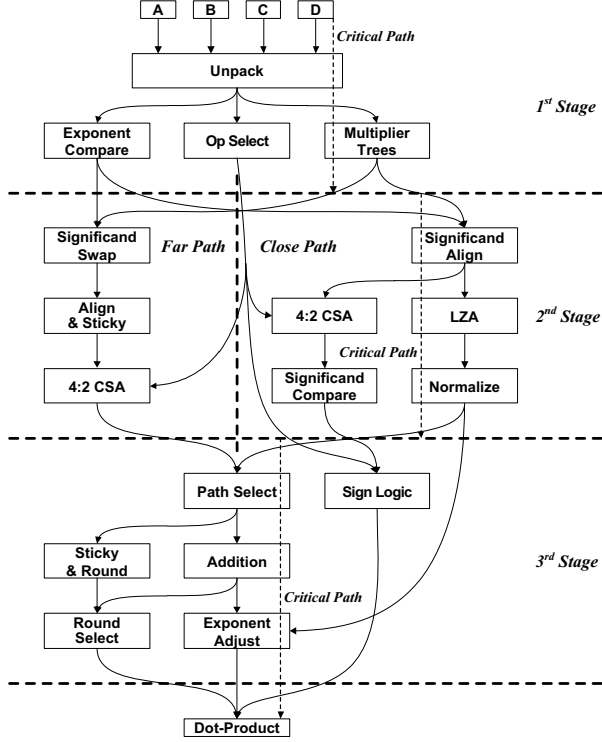


Figure 10. Data flow and critical path of a dual-path fused dot product unit.

where *round_{up}* is the rounding decision of the significand result.

The sign logic determines the final sign bit that is also used in the round logic. The four sign bits of the operands, the input operator, the exponent comparison and the significand comparison are used to determine the sign bit as

$$sign = \begin{cases} AB_{sign}CD_{sign} + AB_{sign}comp_{exp} \\ + AB_{sign}comp_{signif} \\ + CD_{sign}\overline{comp}_{exp}\overline{comp}_{signif} & \text{if } op = add \\ AB_{sign}\overline{CD}_{sign} + AB_{sign}comp_{exp} \\ + AB_{sign}comp_{signif} \\ + \overline{CD}_{sign}\overline{comp}_{exp}\overline{comp}_{signif} & \text{if } op = sub. \end{cases} \quad (15)$$

V. A PIPELINED FUSED DOT PRODUCT UNIT

As is well known, pipelining can improve the throughput of arithmetic units. In order to achieve the proper pipelining for the floating-point fused dot product unit, the arrangement of the components is investigated. Figure 10 shows the data flow and critical path of the dual-path floating-point fused dot product unit. Based on the data flow analysis, the proposed dual-path floating-point fused dot product unit can be split into three stages. The first stage contains unpacking, exponent compare logic and multiplier trees. Since the multiplier trees have a long latency, they occupy most of the latency of the first stage. The second stage consists of the far path and close path logic. The close path logic takes a larger

TABLE II. DOT PRODUCT DESIGN RESULTS COMPARISON

	Traditional	Enhanced	Enhanced + Dual Path	Enhanced + Dual-Path + Pipeline
Area (μm^2)	38,654 (100%)	29,159 (75%)	31,472 (81%)	33,228 (86%)
Latency (ns)	2.54 (100%)	2.14 (84%)	1.87 (74%)	2.01 (79%)
Throughput (1/ns)	0.35 (100%)	0.47 (119%)	0.53 (136%)	1.49 (379%)
Power (mW)	20.77 (100%)	15.17 (73%)	16.16 (78%)	16.94 (82%)

TABLE III. PIPELINED STAGES COMPARISON

	Stage 1	Stage 2	Stage 3
Area (μm^2)	17,484 (53%)	12,143 (36%)	3,601 (11%)
Latency (ns)	0.65 (33%)	0.67 (35%)	0.63 (32%)
Power (mW)	8.96 (53%)	6.41 (38%)	1.57 (9%)

latency than the far path logic so that it becomes the critical path. The third stage contains the path selection, sign logic, significand addition, rounding, and exponent adjust logic. In each pipeline stage, several logic components are performed in parallel and the path that takes the largest latency becomes the critical path. The critical paths of the three stages are

First stage: Unpack $\rightarrow$ Multiplier Trees

Second stage: Close Path Significand alignment $\rightarrow$ LZA $\rightarrow$ Normalization

Third stage: Path Select $\rightarrow$ Significand Addition $\rightarrow$ Exponent Adjust.

The second stage takes the largest latency among the three pipeline stages so that the latency of the second stage becomes the effective latency, which determines the throughput. Due to the latches and control signals between the pipeline stages, the total latency of the pipelined dual-path floating-point fused dot product unit is three times the latency of the second stage. However, the latencies of the three pipeline stages are fairly well balanced so that the throughput is significantly increased compared to the non-pipelined dual-path design.

VI. RESULTS

The floating-point fused dot product unit designs introduced in the previous sections are implemented in Verilog HDL and synthesized with the Nangate 45nm technology standard cell library. In order to evaluate the designs the area, latency, throughput and power consumption are estimated using the simulation. Table II shows the results comparison of the four floating-point fused dot product unit designs. The ratios of the results are based on the proportion to the results of the traditional design.

The enhanced floating-point fused dot product unit applies the new alignment scheme to reduce the shift amount. The early normalization is applied to reduce the size of the significand addition. Also, the proposed four-input LZA reduces the latency by hiding the latency of the reduction tree. As a result, the area and power consumption is reduced by approximately 25%, and the latency is improved by 16% compared to the traditional floating-point fused dot product

unit. The dual-path floating-point fused dot-product unit requires about 5% more area and power consumption than that of the enhanced single path design due to the two path process. However, it eliminates the unnecessary logic in each path so that the latency of the critical path is improved by about 10% compared to the enhanced design.

The proposed pipelined dual-path floating-point fused dot product unit is split into three stages. Table III shows the area, latency and power consumption of the three pipeline stages. Each stage requires latches to maintain the data and control signals between the stages, which increases the area, latency and power consumption. However, the latencies of the three pipeline stages are fairly well balanced so that the throughput is increased to about 2.8 times that of the non-pipelined design.

VII. CONCLUSION

This paper introduces the improved architecture designs and implementations for the floating-point fused two-term dot product unit. Several optimizations are applied to improve the performance: 1) A new alignment scheme, 2) Early normalization and fast rounding, and 3) Four-input LZA. The enhanced floating-point fused dot product unit reduces the area and power consumption by 25% and improves the speed by 15% compared to the traditional floating-point fused dot product unit. Further improvement is achieved by the dual-path algorithm. The dual-path floating-point fused dot product unit consists of a far path and a close path and one path is selected based on the exponent difference. Since the dual-path design eliminates unnecessary logic in each path so that the latency is reduced by about 10% compared to the enhanced single path design. The pipelining is applied to improve the throughput. Based on the data flow analysis, the dual-path floating-point fused dot product unit can be split into three stages. Since the latencies of the three stages are fairly well balanced, the throughput is 2.8 times that of the non-pipelined design.

ACKNOWLEDGMENT

Earl Swartzlander's effort is supported in part by a grant from AMD.

REFERENCES

- [1] *IEEE Standard for Floating-Point Arithmetic*, ANSI/IEEE Standard 754-2008, New York: IEEE, Inc., 2008.
- [2] R. K. Montoye, E. Hokenek, and S. L. Runyon, "Design of the IBM RISC System/6000 Floating-Point Execution Unit," *IBM Journal of Research & Development*, Vol. 34, pp. 59 – 70, 1990.
- [3] E. Hokenek, R. K. Montoye and P. W. Cook, "Second-Generation RISC Floating Point with Multiply-Add Fused," *IEEE Journal of Solid-State Circuits*, vol. 25, pp. 1207 – 1213, 1990.
- [4] T. Lang and J. D. Bruguera, "Floating-Point Fused Multiply-Add with Reduced Latency," *IEEE Transactions on Computers*, Vol. 53, pp. 988 – 1003, 2004.
- [5] H. H. Saleh and E. E. Swartzlander, Jr., "A Floating-Point Fused Add-Subtract Unit," *Proceedings of the 51st IEEE Midwest Symposium on Circuits and Systems*, pp. 519 – 522, 2008.
- [6] J. Sohn and E. E. Swartzlander, Jr., "Improved Architectures for a Fused Floating-Point Add-Subtract Unit," *IEEE Transactions on Circuits and Systems-I*, Vol. 59, pp. 2285 – 2291, 2012.
- [7] H. H. Saleh and E. E. Swartzlander, Jr., "A Floating-Point Fused Dot-Product Unit," *Proceedings of the IEEE International Conference on Computer Design*, pp. 427 – 431, 2008.
- [8] E. E. Swartzlander, Jr. and H. H. Saleh, "FFT Implementation with Fused Floating-Point Operations," *IEEE Transactions on Computers*, Vol. 61, pp. 284 – 288, 2010.
- [9] E. E. Swartzlander, Jr. and H. H. Saleh, "Fused Floating-Point Arithmetic for DSP," *Proceedings of the 42nd Asilomar Conference on Signals, Systems and Computers*, pp. 767 – 771, 2008.
- [10] J. D. Bruguera and T. Lang, "Leading-One Prediction with Concurrent Position Correction," *IEEE Transactions on Computers*, vol. 48, pp. 1083 – 1097, 1999.
- [11] R. Ji, Z. Ling, X. Zeng, B. Sui, L. Chen, J. Zhang, Y. Feng, and G. Luo, Comments on "Leading One Prediction with Concurrent Position Correction," *IEEE Transactions on Computers*, vol. 58, pp. 1726 – 1727, 2009.
- [12] P. Kornerup, "Correcting the Normalization Shift of Redundant Binary Representations", *IEEE Transactions on Computers*, vol. 58, pp. 1435 – 1439, 2009.